

CSCI 6461 | Computer Systems Architecture | TOPIC 3

The Hardware/Software Interface & AArch64 Programmer's Model

Architectural state, registers, memory organization,
and programmer-visible conventions

Dr. J. Burns

Topic 3 Outline

1. Architectural State & the Hardware Contract

- Concrete definition of architectural state vs. microarchitectural structures.
- The AArch64 programmer's model and processor execution modes.

2. AArch64 Register File & Naming Conventions

- The 31 general-purpose registers: 64-bit (x0–x30) vs. 32-bit (w0–w30).
- Special-purpose registers: Zero register (xZR/wZR), stack pointer (sp), program counter (pc).
- Standard ABI register usage: arguments, return values, caller-saved, and callee-saved registers.

3. Memory Organization & Byte Addressing

- Byte-addressable memory, alignment rules, and process address space.

4. Basic Instruction Form

- Instruction syntax, immediate construction, and the basic load/store model.

PART 1

Architectural State & Execution Model

What Is Architectural State?

- **Definition:** The software-visible processor state needed to describe a program's execution at a particular point.
- On a context switch or exception, saving and later restoring the required architectural state allows execution to resume with the same program-visible behavior.
- **Core Components in AArch64:**
 - General-Purpose Registers (x0–x30).
 - Program Counter (pc) and Stack Pointer (sp).
 - Processor state (pstate), including the architectural location of condition flags.
 - System control and configuration registers.
 - Architecturally visible memory state associated with the executing program.

State Checkpointing

Operating systems rely on architectural state to suspend and resume execution while preserving program behavior.

Architectural vs. Microarchitectural State

Architectural State (Visible Contract)

- Defined strictly by the ISA specification.
- Software directly reads/writes this state.
- **Examples:** Registers x0–x30, pc, architectural memory and processor state.

Microarchitectural State (Hidden)

- Internal hardware implementation details.
- Invisible to software correctness.
- **Examples:** Pipeline latches, L1/L2 caches, Branch Target Buffers, rename tables.

Execution boundary. Hardware may execute instructions out of order internally, but retirement must preserve the architectural behavior required by program order.

The AArch64 Execution State

- **AArch64:** The 64-bit execution state introduced in ARMv8-A.
- **64-Bit Address Representation:** General-purpose registers can hold 64-bit pointers, while implemented virtual-address sizes are smaller and configuration-dependent. A64 instructions are fixed 32-bit words aligned to 4-byte boundaries.
- **Distinct A64 Instruction Set:** A64 defines a separate encoding model from A32, removes broad per-instruction conditional execution, and keeps the Program Counter (pc) outside the general-purpose register file.

A64 Design Consequence

A64 is a distinct instruction set from A32, allowing ARM to simplify instruction encoding and remove several older architectural conventions while preserving compatibility through separate execution states on implementations that support them.

PART 2

AArch64 Register Architecture & ABI

General-Purpose Registers: x0–x30

- AArch64 provides **31 general-purpose registers** (x0–x30), each exactly **64 bits wide** (8 bytes).
- **Highly Regular Design:** Most instructions allow general-purpose registers to be used uniformly as sources and destinations, with specific exceptions depending on the instruction encoding.
- **Regular Operand Model:** AArch64 avoids many legacy implicit-register conventions, although some instructions still assign special roles to particular registers.

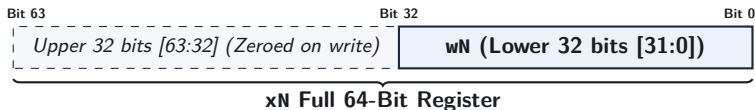


Why Exactly 31 Registers?

Many A64 register fields are 5 bits wide ($2^5 = 32$ encodings). Encoding value 31 denotes either the zero register (xZR/wZR) or the stack pointer (SP/WSP) depending on the instruction class.

32-Bit Register Views: w_0 – w_{30}

- Every 64-bit x register has a corresponding 32-bit view denoted as w_0 through w_{30} .
- w_N represents the **lower 32 bits (bits 31:0)** of the physical register x_N .
- Enables native 32-bit integer arithmetic without requiring separate hardware register files.



Zero-Extension Safety Rule

Writing to a 32-bit w register automatically zeroes bits 63:32 of the corresponding x register.

The Zero Register: `xzr` and `wzr`

- Encoded as register number 31 within standard ALU instruction encodings.
- **Read Behavior:** Always returns constant zero (0 for `xzr`, 32-bit 0 for `wzr`).
- **Write Behavior:** Values written to it are discarded.
- **Architectural Benefit:** Lets several common operations reuse existing instruction formats and supports convenient aliases such as `cmp`.

Common Idioms Using the Zero Register

Assembly Instruction	Equivalent Operation	Purpose
<code>mov x0, xzr</code>	$x_0 \leftarrow 0$	Clear register x_0 to zero
<code>subs xzr, x1, x2</code>	Evaluate $x_1 - x_2 \rightarrow \text{flags}$	Compare x_1 and x_2 (alias: <code>cmp x1, x2</code>)
<code>str xzr, [x1]</code>	$\text{Mem}[x_1] \leftarrow 0$	Zero out a 64-bit memory word

Special-Purpose Registers: `sp`, `pc`, `lr`

Stack Pointer (`sp`)

- Dedicated register holding stack top address.
- Must maintain **16-byte alignment** in the ABI.
- Primarily tracks the current stack location; selected arithmetic and load/store instructions permit `sp` as an operand.

Program Counter (`pc`)

- Represents the address associated with current instruction execution.
- **Cannot** be read/written as data (no `mov x0, pc`).
- PC-relative addresses are formed with instructions such as `adr` and page-relative `adrp`.
- Modified via branch/call instructions.

Link register (`x30` / `lr`). `x30` is a general-purpose register conventionally used as the link register. `bl` and `blr` place the return address in `x30`; `ret` normally returns through it.

The Calling Convention: Caller- vs. Callee-Saved

Caller-Saved Registers (x0–x17)

- Used for arguments and temporary scratch math.
- Caller **must save them** to stack before calls if values are needed later.
- The ABI does not require the callee to preserve their incoming values.

Callee-Saved Registers (x19–x28)

- Preserves long-lived variables across calls.
- If callee modifies these, it **must save and restore** their original contents.
- Guarantees protection of caller state.

Efficiency trade-off. Balancing caller- and callee-saved registers across the ABI minimizes memory traffic and stack spilling during nested subroutine calls.

AArch64 Register File Overview

Register(s)	Main Role
x0–x7	Function arguments and return values
x9–x17	Temporary registers used within short instruction sequences
x19–x28	Long-lived values that must survive subroutine calls
x29 / fp	Frame pointer for stack frames
x30 / lr	Link register: holds a subroutine return address
sp	Stack pointer
pc	Program counter: address of the next instruction
xzr	Zero register: reads as 0; writes are discarded

Know which registers hold data, which support subroutine control, and which special registers appear in common assembly examples. Full ABI preservation rules are developed later when subroutines and the stack are introduced.

Process State: `pstate`

- `pstate` contains selected processor status fields visible to the architecture.
- One part of `pstate` holds the condition flags used by conditional execution.
- Topic 4 explains those flags when they are used for comparisons, branches, and conditional select.

For This Topic

Here, `pstate` is part of the programmer-visible state. Topic 4 explains how N, Z, C, and V are used in assembly control flow.

PART 3

Memory Model, Alignment & Endianness

Memory Organization & Byte Addressing

- Memory is byte-addressed and exposed to software as a sequence of **8-bit bytes**.
- Addresses are represented in 64-bit registers, although implemented virtual-address ranges are smaller than the full 64-bit space.
- Data operands span consecutive byte addresses scaling by standard powers of two:

Data Type	Size (Bytes)	Size (Bits)	AArch64 Architecture Term
Byte	1	8-bit	Byte (b)
Halfword	2	16-bit	Halfword (h)
Word	4	32-bit	Word (w)
Doubleword	8	64-bit	Doubleword (x)
Quadword	16	128-bit	Quadword / Vector element (q)

64-Bit Pointer Translation

While pointers are 64 bits wide, implemented virtual and physical address sizes are smaller and configuration-dependent; common systems use ranges such as 48 or 52 bits.

Memory Alignment Rules

- An access to an n -byte operand is **naturally aligned** if $\text{Address} \pmod n = 0$.
- **Alignment Rules:** Word (4B boundary), Doubleword (8B boundary), Instruction fetch (4B boundary).

Aligned Access (Single Transaction)

64-bit Word [Bytes 0–7]

Unaligned Access (Crosses Boundary)

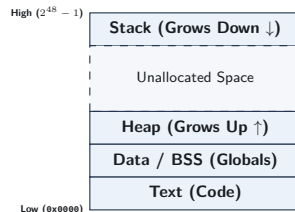
Word Part 1

Word Part 2

Alignment penalties. Many ordinary accesses to Normal memory can be unaligned, although an unaligned access may require additional memory transactions or cycles. Some access types, memory attributes, or configured environments require alignment and can generate an alignment fault.

The Process Address Space Layout

- **Text (Code):** Read-only machine instructions.
- **Data / BSS:** Globally accessible and static variables.
- **Heap:** Dynamically allocated memory via malloc; grows **upward** toward higher addresses.
- **Stack:** Local function variables and saved caller state; conventionally grows **downward** toward lower addresses.



Simplified Address-Space Model

Modern processes also contain shared libraries, memory-mapped files, anonymous mappings, guard pages, and ASLR-created gaps. Excessive recursion typically reaches a stack limit or guard page and faults.

PART 4

Foundational Instruction Classes

Data Processing: Register and Immediate Operands

- AArch64 data-processing instructions usually name the **destination** and source operands explicitly.
- Operands may be registers or **immediate constants**; x and w register names select 64-bit and 32-bit views.

Example	Purpose
<code>add x0, x1, x2</code>	Add two 64-bit register values
<code>sub w3, w4, w5</code>	Subtract using 32-bit register views
<code>add x0, x1, #16</code>	Add an immediate constant

Architectural View

These instructions read architectural register state, perform an operation, and write a result back to a destination register.

Immediate Constants: Building Values in Registers

- Every AArch64 instruction is 32 bits wide, so an arbitrary 64-bit constant cannot always be encoded directly in one instruction.
- AArch64 can build larger constants from smaller pieces using instructions such as `movz` and `movk`.
- The detailed immediate encodings are not important yet; the key point is that **large constants may require multiple instructions**.

Example: Constructing 0x1234abcd

<code>movz x0, #0xabcd</code>	start with the low 16 bits
<code>movk x0, #0x1234, lsl #16</code>	insert the next 16-bit portion

Architectural constraint. Immediate values are part of the instruction itself. Their size and placement are therefore constrained by the fixed instruction width, unlike values already stored in registers or memory.

Memory Access: Loads and Stores

- AArch64 is a **load/store architecture**: arithmetic and logic operate primarily on registers, while memory is accessed explicitly.
- `ldr` copies a value from memory into a register; `str` copies a register value to memory.
- Brackets identify a register being used as a memory address, as in `[x1]`.

Example	Transfer	Meaning
<code>ldr x0, [x1]</code>	64 bits	load from the address in x1
<code>str x0, [x1]</code>	64 bits	store to the address in x1
<code>ldr w0, [x1]</code>	32 bits	load a 32-bit value into w0

Later Detail

Topic 5 develops offsets, indexed forms, writeback, transfer sizes, and the main AArch64 addressing modes used for memory access. Here, the goal is simply to recognize explicit register–memory transfers.

Branching and Control Flow

- Normally, execution advances to the next instruction; a **branch** changes the address used for the next instruction.
- Branches provide the basic mechanism for decisions, loops, function calls, and returns.

Example	Role
b loop	unconditional branch
b.eq done	conditional branch
bl function	function call
ret	return from a function

Connection to Architectural State

A taken branch changes the next pc; conditional control flow may also depend on condition flags or a tested register value. Topic 4 develops these mechanisms in detail.

SUMMARY & ASSESSMENT

Review and Self-Test Questions

Topic 3: Summary

- **Architectural State:** The programmer-visible contract includes general registers (x0–x30), processor state, stack pointer (sp), program counter (pc), and byte-addressed memory.
- **Register Structure:** The 64-bit x0–x30 registers have corresponding 32-bit w0–w30 views; writing a w register clears the upper 32 bits of the corresponding x register.
- **ABI Roles:** Register use is constrained by calling-convention rules, including argument/return registers and caller- vs. callee-saved registers.
- **Memory Model:** AArch64 exposes byte-addressed memory with alignment requirements and explicit load/store instructions.
- **Instruction Classes:** Basic data processing, immediate construction, explicit loads/stores, and branching illustrate how AArch64 instructions transform architectural state.

Self-Test Questions: State & Register Mechanics

- ❶ **Architectural vs. Microarchitectural State:** Why are CPU branch prediction tables and physical register rename structures classified as microarchitectural state rather than architectural state?
- ❷ **32-bit Register Write Semantics:** Suppose register x5 initially contains `0xffffffff_ffffffff`. If the instruction `mov w5, #0x1234` is executed, what exact 64-bit value resides in x5 post-execution?
- ❸ **Zero Register Application:** Why does AArch64 provide a dedicated zero register (`xzr/wzr`) instead of permanently assigning a general-purpose register (like x0) to hold zero?
- ❹ **Calling Convention Responsibility:** If function `foo()` calls `bar()`, and `foo()` needs to preserve values across the call in registers x3 and x20, which function is responsible for saving each register to the stack?

Self-Test Questions: Memory & Instruction Mechanics

- ❶ **Memory Model:** Why does AArch64 use explicit load/store instructions rather than allowing most arithmetic instructions to read directly from memory?
- ❷ **Alignment:** Why can naturally aligned accesses be simpler for hardware than accesses that cross an alignment boundary?
- ❸ **Immediate Construction:** Why may a 64-bit constant require more than one AArch64 instruction using `movz/movk`?
- ❹ **Branching:** What programmer-visible state determines which instruction executes next, and how does a taken branch change it?

Looking Ahead

Next Lecture Preview

Our Next Topic: AArch64 Assembly Programming: Arithmetic, Logic, & Control Flow

- **ALU & Shifted Operands:** Arithmetic and shifts using AArch64 data-processing instructions.
- **Bitwise Logic & Field Extraction:** Manipulating bitfields using masks, bit clears (`bic`), and bitfield extracts (`ubfx`).
- **Condition Flags & Comparisons:** Using N, Z, C, and V for comparisons and conditional control flow.
- **Control Flow & Conditional Selection:** Translating loops and nested conditionals while using `csel` to reduce reliance on conditional branches for suitable expressions.